

Phase 1 SSVI Workbook

Positioning

This document is written as a junior quant-dev exercise for the repository vol-surface-engine. The goal is to finish phase 1 cleanly: a European SSVI calibration that is understandable, testable, and usable as a warm start for the later American-option calibration loop.

You are not trying to solve the full business problem today. You are trying to produce a clean phase-1 asset:

- a dedicated runner for European calibration
- explicit diagnostics for fit quality
- one synthetic regression test that proves the calibration stack works when the data is well-behaved

The main discipline for the day is simple: do not start with Nelder-Mead, and do not jump to the American fitter. First make the European layer trustworthy.

Business goal of phase 1

The firm ultimately cares about American options. That means the final pricing fit must happen through an American pricing model, not just through a European implied-vol surface. Still, the European calibration matters because it gives you three useful things:

- a smooth, compact, arbitrage-controlled representation of the smile
- a warm start for the later American fit
- a cleaner separation between data issues and pricing-model issues

In other words, phase 1 is not the final answer. It is the initialization layer. If phase 1 is messy, every later debugging session becomes ambiguous: is the issue in the data, in the parameterization, in Dupire, in the PDE, or in the American early-exercise step? A clean phase 1 removes one entire class of uncertainty.

What "done" means today

At the end of the session, you want four outcomes:

- `apps/calibrate_spx_main.cpp` exists as a dedicated European calibration runner
- `include/calibration/FitDiagnostics.hpp` and `src/calibration/FitDiagnostics.cpp` expose a small diagnostics helper
- `tests/test_ssvi_synthetic.cpp` proves that synthetic SSVI data can be recovered by the fitter
- you can answer the business question: when the real-data fit sits on the arbitrage boundary, is that likely due to optimizer weakness or model rigidity?

Technical warning before coding

Your current code already calibrates SSVI inside `src/main.cpp`. That is useful as a prototype, but it is not a clean phase-1 deliverable. The current weaknesses are:

- validation logic and live calibration logic are mixed together
- diagnostics are too thin to explain why the fit behaves poorly
- there is no synthetic regression test for the calibration stack
- the current global parameterization $\theta(T) = \nu * T$ is very restrictive

This matters because a boundary-hitting calibration is often not an optimizer bug. It is frequently a sign that the model wants more freedom than the parameterization allows.

Working order for today

Use this order and do not improvise around it:

- first: write the synthetic calibration test
- second: write the diagnostics helper
- third: extract the dedicated SPX runner
- fourth: run the real-data calibration and inspect the diagnostics

The reason for this order is that you want one trusted reference case before reading too much into the real market fit.

Exercise 1 — Synthetic regression test

File

tests/test_ssvi_synthetic.cpp

What this file should do

This is the most important exercise of the day. The job of this file is to answer the question: if the world truly follows my SSVI parameterization, can my fitter recover the parameters? If the answer is no, you should not trust any diagnosis based on real market data.

The idea is very simple:

- choose a known parameter set p^*
- generate a grid of implied vols directly from `impliedVolSSVI`
- feed that grid into `calibrateSSVI`
- verify that the recovered parameters are essentially equal to p^*
- verify that the fit error is numerically tiny

Technical subtleties

- Use synthetic data with no noise for the first test. This isolates the optimizer and the parameterization from market microstructure.
- Use the same k-grid and T-grid shapes you expect in production. A test that is too toy-like can pass while production still fails for structural reasons.
- Check both parameter recovery and output-surface recovery. In a nonlinear model, two parameter sets can produce very similar smiles over a narrow domain.
- Keep the tolerances tight but not absurd. The current implementation recovers the synthetic parameters to around machine precision, so a 1e-6 style tolerance is realistic.

Pseudocode

```
choose true_params
build k_grid
build T_grid
for each T and k:
    iv_market(T, k) = impliedVolSSVI(k, T, true_params)

fitted = calibrateSSVI(k_grid, T_grid, iv_market)
assert calibration succeeded
```

```

diagnostics = diagnoseSSVIFit(k_grid, T_grid, iv_market, fitted)
assert |fitted.rho - true.rho| < tolerance
assert |fitted.eta - true.eta| < tolerance
assert |fitted.gamma - true.gamma| < tolerance
assert |fitted.nu - true.nu| < tolerance
assert diagnostics.rmse_total is tiny

```

Corrigé C++

```

#include "calibration/FitDiagnostics.hpp"
#include "calibration/SSVI.hpp"

#include <Eigen/Dense>

#include <cassert>
#include <cmath>
#include <iostream>
#include <variant>
#include <vector>

int main() {
    using vse::calibration::FitDiagnostics;
    using vse::calibration::SSVIParams;

    const SSVIParams true_params{
        .rho = -0.30,
        .eta = 0.50,
        .gamma = 0.30,
        .nu = 0.04,
    };

    const std::vector<double> k_grid{ -0.4, -0.2, -0.1, 0.0, 0.1, 0.2, 0.4 };
    const std::vector<double> T_grid{ 0.25, 0.50, 1.00, 2.00 };

    Eigen::MatrixXd iv_market(T_grid.size(), k_grid.size());
    for (std::size_t i = 0; i < T_grid.size(); ++i) {
        for (std::size_t j = 0; j < k_grid.size(); ++j) {
            iv_market(i, j) = vse::calibration::impliedVolSSVI(k_grid[j], T_grid[i], true_params);
        }
    }

    auto result = vse::calibration::calibrateSSVI(k_grid, T_grid, iv_market);
    assert(std::holds_alternative<SSVIParams>(result));

    const SSVIParams fitted = std::get<SSVIParams>(result);
    const FitDiagnostics diag = vse::calibration::diagnoseSSVIFit(k_grid, T_grid, iv_market, fitted);

    auto close = [](double x, double y, double tol) {
        return std::abs(x - y) < tol;
    };

    assert(close(fitted.rho, true_params.rho, 1e-6));
    assert(close(fitted.eta, true_params.eta, 1e-6));
    assert(close(fitted.gamma, true_params.gamma, 1e-6));
    assert(close(fitted.nu, true_params.nu, 1e-6));

    assert(diag.rmse_total < 1e-10);
    assert(diag.eta_ratio > 0.0);
    assert(diag.eta_ratio < 1.0);

    std::cout << "Synthetic SSVI calibration test: OK\n";
    return 0;
}

```

Exercise 2 — Diagnostics helper

Files

- include/calibration/FitDiagnostics.hpp
- src/calibration/FitDiagnostics.cpp

What these files should do

These files should expose a tiny diagnostics layer. The layer is not supposed to change the calibration. It is supposed to explain it.

For phase 1, the minimum useful outputs are:

- total RMSE on the calibration grid
- maximum absolute error on the calibration grid
- one slice summary per maturity
- $\text{eta_ratio} = \text{eta} / \text{eta_max}(\text{gamma}, \text{rho})$

The eta ratio is especially important. If the real-data fit keeps finishing with eta_ratio extremely close to 1, you want to see that explicitly rather than infer it from raw parameters.

Technical subtleties

- Diagnostics must be computed on the same k-grid and T-grid used in calibration.
- Keep the helper pure: no I/O, no printing, no file reads.
- The current repository keeps etaMax internal to SSVI.cpp. For today, duplicating the formula in the diagnostics helper is acceptable if it lets you move fast. Later, you can expose a shared public helper.
- A low total RMSE can hide one bad maturity. That is why the per-slice summaries matter.

Pseudocode

```
check dimensions
for each maturity i:
    initialize slice error accumulators
    for each strike j:
        model_iv = impliedVolSSVI(k_j, T_i, params)
        error = model_iv - market_iv(i, j)
        accumulate total and slice metrics
    finalize slice rmse
compute overall rmse
compute eta_ratio = eta / eta_max(gamma, rho)
return diagnostics
```

Corrigé C++ — header

```
#ifndef FITDIAGNOSTICS_HPP
#define FITDIAGNOSTICS_HPP

#include "calibration/SSVI.hpp"

#include <Eigen/Dense>

#include <span>
#include <vector>

namespace vse::calibration {

struct SliceFitSummary {
    double T{};
    int    n_points{};
    double rmse{};
    double max_abs_error{};
};

struct FitDiagnostics {
    double rmse_total{};
    double max_abs_error{};
    double eta_ratio{};
    std::vector<SliceFitSummary> slices;
};

}
```

```

FitDiagnostics diagnoseSSVIFit(
    std::span<const double> k_grid,
    std::span<const double> T_grid,
    const Eigen::MatrixXd& iv_market,
    const SSVIPParams& params
);

} // namespace vse::calibration

#endif

```

Corrigé C++ — implementation

```

#include "calibration/FitDiagnostics.hpp"

#include <algorithm>
#include <cmath>
#include <limits>
#include <stdexcept>

namespace vse::calibration {
namespace {

double phiSqSupFactorLocal(double gamma) noexcept {
    if (gamma > 0.5) return std::numeric_limits<double>::infinity();
    const double a = 1.0 - 2.0 * gamma;
    if (a == 0.0) return 1.0;
    return std::pow(a, a) / std::pow(1.0 + a, 1.0 + a);
}

double etaMaxBoundLocal(double gamma, double rho) noexcept {
    const double ar = 1.0 + std::abs(rho);
    return std::min(4.0 / ar, 2.0 / std::sqrt(phiSqSupFactorLocal(gamma) * ar));
}

} // namespace

FitDiagnostics diagnoseSSVIFit(
    std::span<const double> k_grid,
    std::span<const double> T_grid,
    const Eigen::MatrixXd& iv_market,
    const SSVIPParams& params
) {
    if (static_cast<std::size_t>(iv_market.cols()) != k_grid.size()) {
        throw std::invalid_argument("diagnoseSSVIFit: k-grid size mismatch");
    }
    if (static_cast<std::size_t>(iv_market.rows()) != T_grid.size()) {
        throw std::invalid_argument("diagnoseSSVIFit: T-grid size mismatch");
    }

    FitDiagnostics out;
    out.slices.reserve(T_grid.size());

    double sse_total = 0.0;
    int n_total = 0;

    for (std::size_t i = 0; i < T_grid.size(); ++i) {
        double sse_slice = 0.0;
        double max_abs_slice = 0.0;

        for (std::size_t j = 0; j < k_grid.size(); ++j) {
            const double model_iv = impliedVolSSVI(k_grid[j], T_grid[i], params);
            const double err = model_iv - iv_market(i, j);
            const double abs_err = std::abs(err);

            sse_total += err * err;
            sse_slice += err * err;
            out.max_abs_error = std::max(out.max_abs_error, abs_err);
            max_abs_slice = std::max(max_abs_slice, abs_err);
            ++n_total;
        }

        out.slices.push_back(SliceFitSummary{
            .T = T_grid[i],
            .n_points = static_cast<int>(k_grid.size()),
            .rmse = std::sqrt(sse_slice / static_cast<double>(k_grid.size())),
            .max_abs_error = max_abs_slice,
        });
    }
}

```

```

    }

    out.rmse_total = std::sqrt(sse_total / static_cast<double>(n_total));
    out.eta_ratio = params.eta / etaMaxBoundLocal(params.gamma, params.rho);
    return out;
}

} // namespace vse::calibration

```

Exercise 3 — Dedicated SPX calibration runner

File

apps/calibrate_spx_main.cpp

What this file should do

This file should become the clean command-line runner for phase 1. It should not contain every experiment you ever made. It should do one job well:

- load curated SPX data
- compute implied vols from quotes
- build maturity slices
- interpolate onto a common k-grid
- run calibrateSSVI
- print meaningful diagnostics

Technical subtleties

- Use forward log-moneyness $k = \log(K / F)$, not spot log-moneyness. This is the right coordinate for smile work.
- Keep the first live version simple: flat $r = 0$ and $q = 0$ are acceptable, but print that assumption clearly.
- Filter slices before interpolation. A common-grid interpolation built from one-sided or very sparse slices is unstable and hard to interpret.
- The purpose of this runner is transparency. Avoid hiding logic inside an opaque helper until the workflow is trusted.

Pseudocode

```

load market data
for each call and put quote:
    compute mid
    skip bad quotes
    compute forward F and  $k = \log(K / F)$ 
    invert implied vol
    keep valid points near ATM

group points by maturity
keep slices with enough points and both wings
compute common k-range intersection
build a common k-grid
interpolate each slice onto that grid
calibrate SSVI
compute diagnostics
print parameter values and per-maturity errors

```

Corrigé C++

```
#include "calibration/FitDiagnostics.hpp"
#include "calibration/ImpliedVol.hpp"
#include "calibration/SSVI.hpp"
#include "data/MarketData.hpp"
#include "pricing/blackscholes.hpp"

#include <Eigen/Dense>

#include <algorithm>
#include <cmath>
#include <iostream>
#include <limits>
#include <map>
#include <string>
#include <utility>
#include <variant>
#include <vector>

namespace {

struct IVPoint {
    double k;
    double T;
    double iv;
};

double interpolateSlice(const std::vector<std::pair<double, double>>& pts, double k) {
    if (pts.empty()) return -1.0;
    if (k < pts.front().first || k > pts.back().first) return -1.0;

    auto it = std::lower_bound(
        pts.begin(), pts.end(), std::make_pair(k, -std::numeric_limits<double>::infinity())
    );
    if (it == pts.begin()) return it->second;
    if (it == pts.end()) return pts.back().second;

    auto hi = it;
    auto lo = std::prev(it);
    const double w = (k - lo->first) / (hi->first - lo->first);
    return lo->second + w * (hi->second - lo->second);
}

} // namespace

int main(int argc, char** argv) {
    const std::string folder = (argc > 1) ? argv[1] : "data/csv/SPX";
    const std::string today = (argc > 2) ? argv[2] : "2026-09-12";

    const auto md = vse::data::MarketData::load(folder, today);
    const double S0 = md.spot > 0.0 ? md.spot : 7656.98;
    const double r = 0.0;
    const double q = 0.0;

    std::cout << "Phase 1 European SSVI calibration\n";
    std::cout << "Data folder: " << folder << "\n";
    std::cout << "Assumptions: flat r = " << r << ", flat q = " << q << "\n\n";

    std::vector<IVPoint> pts;
    auto add_iv = [&](const vse::data::OptionQuote& quote, bool is_call) {
        const double mid = quote.mid();
        if (mid <= 0.0) return;

        const double F = S0 * std::exp((r - q) * quote.T);
        const double k = std::log(quote.K / F);
        if (std::abs(k) > 0.50) return;

        vse::pricing::BSPParams bp{S0, quote.K, quote.T, r, q, 0.20};
        const auto type = is_call
            ? vse::calibration::OptionType::Call
            : vse::calibration::OptionType::Put;

        const auto iv = vse::calibration::impliedVol(mid, bp, type, 1e-4, 5.0, 1e-8, 200);
        if (std::holds_alternative<vse::calibration::IVResult>(iv)) {
            pts.push_back({k, quote.T, std::get<vse::calibration::IVResult>(iv).sigma});
        }
    };

    for (const auto& qte : md.calls) add_iv(qte, true);
}
```

```

for (const auto& qte : md.puts) add_iv(qte, false);

if (pts.empty()) {
    std::cerr << "No valid implied-vol points.\n";
    return 1;
}

std::map<double, std::vector<std::pair<double, double>>> by_T;
for (const auto& p : pts) {
    by_T[p.T].push_back({p.k, p.iv});
}

std::vector<double> T_grid;
std::vector<std::vector<std::pair<double, double>>> slices;

for (auto& [T, slice] : by_T) {
    if (slice.size() < 8) continue;
    std::sort(slice.begin(), slice.end());
    if (T < 0.08) continue;
    if (slice.front().first > -0.05) continue;
    if (slice.back().first < 0.05) continue;

    T_grid.push_back(T);
    slices.push_back(slice);
}

if (T_grid.empty()) {
    std::cerr << "No bilateral slices after filtering.\n";
    return 1;
}

double k_lo = -1.0;
double k_hi = 1.0;
for (std::size_t i = 0; i < slices.size(); ++i) {
    k_lo = std::max(k_lo, slices[i].front().first);
    k_hi = std::min(k_hi, slices[i].back().first);
}

if (k_lo >= k_hi) {
    std::cerr << "Empty common k-range.\n";
    return 1;
}

constexpr int NK = 11;
std::vector<double> k_grid(NK);
for (int j = 0; j < NK; ++j) {
    k_grid[j] = k_lo + j * (k_hi - k_lo) / static_cast<double>(NK - 1);
}

Eigen::MatrixXd iv_market(T_grid.size(), NK);
std::vector<double> T_ok;
std::vector<Eigen::VectorXd> rows;

for (std::size_t i = 0; i < slices.size(); ++i) {
    Eigen::VectorXd row(NK);
    bool ok = true;
    for (int j = 0; j < NK; ++j) {
        const double iv = interpolateSlice(slices[i], k_grid[j]);
        if (iv < 0.0) {
            ok = false;
            break;
        }
        row[j] = iv;
    }
    if (ok) {
        T_ok.push_back(T_grid[i]);
        rows.push_back(row);
    }
}

if (rows.empty()) {
    std::cerr << "Interpolation produced no full slices.\n";
    return 1;
}

Eigen::MatrixXd iv_ok(rows.size(), NK);
for (std::size_t i = 0; i < rows.size(); ++i) {
    iv_ok.row(i) = rows[i];
}

const auto result = vse::calibration::calibrateSSVI(k_grid, T_ok, iv_ok);
if (!std::holds_alternative<vse::calibration::SSVIPParams>(result)) {

```

```

        std::cerr << "SSVI calibration failed.\n";
        return 1;
    }

    const auto params = std::get<vse::calibration::SSVIParams>(result);
    const auto diag = vse::calibration::diagnoseSSVIFit(k_grid, T_ok, iv_ok, params);

    std::cout << "Valid IV points: " << pts.size() << "\n";
    std::cout << "Maturities kept: " << T_ok.size() << "\n";
    std::cout << "Common k-range: [" << k_lo << ", " << k_hi << "]\n\n";

    std::cout << "Fitted parameters\n";
    std::cout << "  rho  = " << params.rho << "\n";
    std::cout << "  eta  = " << params.eta << "\n";
    std::cout << "  gamma = " << params.gamma << "\n";
    std::cout << "  nu   = " << params.nu << "\n";
    std::cout << "  eta / eta_max = " << diag.eta_ratio << "\n";
    std::cout << "  total RMSE    = " << diag.rmse_total << "\n";
    std::cout << "  max abs error = " << diag.max_abs_error << "\n\n";

    std::cout << "Per-maturity diagnostics\n";
    for (const auto& s : diag.slices) {
        std::cout << "  T=" << s.T
                  << "  n=" << s.n_points
                  << "  rmse=" << s.rmse
                  << "  max_abs=" << s.max_abs_error << "\n";
    }

    return 0;
}

```

Minimal acceptance checklist

Before you call phase 1 clean, check all of the following:

- the synthetic test passes
- the runner compiles and executes on SPX data
- the runner prints eta / eta_max explicitly
- the runner prints at least one error metric per maturity
- there is no SSVI business logic left hidden inside a giant sandbox main

How to interpret the boundary issue

Once the diagnostics exist, interpret the outcome with discipline:

- If the synthetic test passes with tiny error and recovered parameters, the calibration stack is basically sound.
- If the real-data fit still lands near eta / eta_max = 1, the first suspect is the model setup, not the optimizer.
- The most likely structural issue is the rigidity of $\theta(T) = \nu * T$.
- Secondary suspects are the objective choice, the data filters, and the slice interpolation pipeline.

This is the exact reason you do phase 1 cleanly before American fitting. You want one trustworthy European layer before adding Dupire, PDE discretization, and early exercise.

Recommended next step after this workbook

Do not jump directly to a custom optimizer. First run the real-data calibration with the new diagnostics and write down the answer to three questions:

- which maturities are fitted badly?
- is the error mostly near the wings or near ATM?
- how close is eta / eta_max to 1?

Only after that should you decide whether the next change is:

- a richer $\theta(T)$ parameterization
- a different objective weighting
- better market-data filtering
- or, only later, an optimizer comparison.